Secure By Design

EDUCATE. EMPOWER. SECURE THE FUTURE.

COURSE 1.15 / COMPANION WORKBOOK

Keep the living inventory on SBD-MGMT01

Component inventory, discovery sources & monthly reconciliation

Turn course 1.13's fleet snapshot into a living component inventory for the whole network: build SBD-MGMT01 on VLAN 70 as a keeper that reaches into nothing, record every component once with an accountable owner, check the record against what the directory, DHCP, WSUS and SBD-REPO01 already know, and reconcile it every month so drift becomes a finding.

MATERIALS EDITION 1.0 / COURSE 1.15

Updated 14 October 2026 · sbd-management-server

NAME _____ / DATE _____

START HERE / YOUR LEARNING PATH

Table of contents

Read each lesson alongside the interactive guide and the video. Use the record tables to write down what you actually observe — the workbook is your evidence, not the guide.

Before you start

01 Design the inventory and its keeper

02 Build and harden SBD-MGMT01

03 Build the living inventory

04 Feed it from what the network already knows

05 Reconcile the inventory and prove it

Course review and sign-off

> **HOW TO USE IT** Each lesson page has the same shape: what you will learn, the build steps, the commands that verify them, and a record table. Fill the table from your own environment — never from the sample values.

Before you start

Course 1.13 handed over a snapshot of the fleet; this course makes the whole network's component inventory a living record on SBD-MGMT01 — NIST SP 800-53 CM-8 in practice. The server keeps the record and reaches into nothing: the directory, the DHCP leases, the WSUS computer list, SBD-REPO01's records and the baseline sessions it is checked against arrive through the administrators' existing sessions. Column names and the monthly cadence are sample values; the system owner decides.

Prerequisites

- ☐ Complete courses 1.1–1.14: every system those courses built is running, course 1.13's fleet inventory is filed, and SBD-WSUS01 serves all four update rings.
- ☐ Provision SBD-MGMT01 (10.0.70.10, VLAN 70) as a Windows Server virtual machine with a separate data volume (D: in this sample) for the record.
- ☐ Keep the records earlier courses produced within reach: the service-path register, course 1.14's ring table and WSUS computer list, course 1.9's snapshot catalog, each course's baseline sessions, and the exception record.

What this course builds

- Define the inventory's boundary, columns and owners so every component is counted once and accountable.
- Build SBD-MGMT01 as a hardened keeper administered only from the Tier 1 seat.
- Gather discovery sources through existing administration paths and know what each one proves.
- Reconcile recorded and discovered components every month, with a planted test proving detection works.

Systems in scope

RECORD	YOUR OBSERVATION
SBD-MGMT01 — name, address, OS build	

© 2026 Secure By Design Team LLC.

LESSON 01 / ABOUT 25 MIN

Design the inventory and its keeper

> **WHY THIS MATTERS** Decide what the component inventory must hold before SBD-MGMT01 exists: every component inside the boundary counted once, and the columns that make each one accountable. Then name the records that can contradict it, and give it a keeper that holds the record without reaching into anything else.

In this lesson you will

- Draw the inventory's boundary and list every component in it.
- Choose the columns that make each component accountable.
- Name the discovery sources and place SBD-MGMT01 under the tier model.

Follow the build

1. Draw the boundary as CM-8 asks: every component of this network, each counted once, and nothing from outside it. That is every server and controller (SBD-ROOTCA01 included, offline in the vault), the three administration seats, each of the 50 PCs, the core and the VLAN 80 access switch, and the seven hypervisor hosts (SBD-HV01–SBD-HV07) on which Inside the Network places the virtual machines. The connected-side systems that fill the media stay out; the crossing records name them.
2. Choose the columns, one per attribute: name, type, role, operating system and version, domain membership, address and VLAN, OU, tier, location (host and rack), update source (a WSUS ring, an SBD-REPO01 snapshot, or by hand at a ceremony), latest baseline, the course that built it, an accountable owner by role (CM-8(4)), and status — planned, active or retired. Record the systems the design names but no course has built yet, SBD-EPO01, SBD-NESSUS01 and the four VLAN 40 servers, as planned, so a reconciliation never mistakes them for missing.
3. Name the records that can contradict it and what each one proves: the directory shows what joined the domain, the DHCP leases what is on VLAN 80 now, WSUS's computer list what reports for updates and in which ring, SBD-REPO01's catalog and access log which Linux servers take packages, and the baseline sessions which hosts have a measured state. Any disagreement is a finding, and lesson 5 makes the comparison a monthly check.
4. Place the keeper as Inside the Network does: SBD-MGMT01, Windows Server at 10.0.70.10 on VLAN 70 beside SBD-REPO01, a Tier 1 member server administered only from SBD-ADMIN-SRV01 (rule A02). The model leaves its listeners to define: define them as management only, and give it no collection path of its own — source data arrives through the administrators' existing sessions, so the server that holds the record can't reach, or be used to reach, anything else. Its domain rows (F01–F03, F06 in Ring 3, F09, F17 and F18) already list it.

Check your result

```
# On SBD-ADMIN-SRV01: start the course record with the inventory's columns
New-Item -ItemType Directory -Force C:\Evidence\management-server | Out-Null
'Name,Type,Role,OS,Domain,Address,VLAN,OU,Tier,Location,UpdateSource,Baseline,BuiltIn,Owne
r,Status' | Set-Content C:\Evidence\management-server\inventory-columns.csv
# SBD-MGMT01 isn't registered yet; its neighbour on VLAN 70 is
Resolve-DnsName SBD-MGMT01.sbd.test -ErrorAction SilentlyContinue   # expected: no answer
Resolve-DnsName SBD-REPO01.sbd.test | Select-Object Name, IPAddress   # 10.0.70.20
```

WHAT SUCCESS LOOKS LIKE The boundary lists every component once, with the planned systems marked; the columns cover identity, place, tier, update source, baseline and an accountable owner; each discovery source is named with what it proves; and SBD-MGMT01's placement, administration and lack of any collection path are written down.

Your record

Evidence to keep: Keep the boundary list, the column definitions, the source table with what each source proves, and SBD-MGMT01's placement and administration decision.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why record the hypervisor hosts when every server is a virtual machine?

REMEMBER Every component inside the boundary, counted once — hosts and network devices included. Columns that make each component accountable, owner included. Sources that can contradict the record, and a keeper that reaches into nothing.

NIST · SP 800-53 Rev. 5 security controls (CM-8 System Component Inventory)

LESSON 02 / ABOUT 35 MIN

Build and harden SBD-MGMT01

> **WHY THIS MATTERS** Build the inventory's keeper as a small, quiet member server, joined into the Servers OU so the hardening, tier and update policies apply from its first boot. It answers only the Tier 1 seat, and the record sits on its own volume behind permissions that admit administrators alone.

In this lesson you will

- Build SBD-MGMT01 in its place with the network's policies applied from first boot.
- Limit its administration to SBD-ADMIN-SRV01 and prove the limit.
- Prepare the record's folders, permissions and version history.

Follow the build

1. Build SBD-MGMT01 as a Windows Server virtual machine — on SBD-HV07 beside SBD-REPO01, where Inside the Network places it — with the hypervisor's time synchronization turned off, so the domain is its only clock, a separate data volume for the record (D: in this sample), the static address 10.0.70.10/24 with the core's VLAN 70 interface as gateway (10.0.70.1 in this sample), and both domain controllers for DNS. Join it straight into the Servers OU, so course 1.5's STIG policies, course 1.8's tier logon policies, course 1.10's event-forwarding GPO (linked to the Servers OU in course 1.10) and course 1.14's Ring 3 update policy apply from its first boot.
2. Check that it took its place: time from a domain controller, as course 1.3 set up, and an update policy that names SBD-WSUS01 and Ring 3, with SBD-WSUS01 listing it there. Add it to SBD Event Sources from AdminJumpBox01, widen the collector's WinRM scope to 10.0.70.10 (F17) from SBD-ADMIN-SRV01, and restart it so its security events reach SBD-SYSLOG01.
3. Close it to everything but its administration: Windows Defender Firewall on in every profile, and the existing Remote Desktop and Windows Remote Management rules scoped to SBD-ADMIN-SRV01 (10.0.60.20), because allow rules add up and a new, narrower rule would not restrict the broad ones. Install no other role and no agent; course 1.8's Servers OU policy already denies Tier 0 accounts here.
4. Prepare the record on D:: an Inventory folder with inheritance removed and full control for SYSTEM and Administrators only — the volume's inherited defaults let ordinary users read and create files — holding current, history, sources and findings folders. Every published version is copied into history with its SHA-256 in a hash list, and every change gets a line in a change log: who, what, why, and the change record it answers.
5. Prove the closure: a remote session from SBD-ADMIN-SRV01 opens, the same attempt from SBD-ADMIN-PC01 fails at the core because rule A02 names only the Tier 1 seat, and the firewall's remote-address lists show only 10.0.60.20 for management.

Check your result

```
# In a session on SBD-MGMT01 from SBD-ADMIN-SRV01
([adsisearcher]"(&(objectCategory=computer)(name=$env:COMPUTERNAME))").FindOne().Path
# ...OU=Servers,OU=SBD,DC=sbd,DC=test
w32tm /query /source   # a domain controller, never the local clock
Get-ItemProperty HKLM:\SOFTWARE\Policies\Microsoft\Windows\WindowsUpdate | Select-Object
WUServer, TargetGroup   # Ring 3 - Member servers
```

```
Get-NetFirewallRule -DisplayGroup 'Remote Desktop', 'Windows Remote Management' -Enabled
True | Get-NetFirewallAddressFilter | Select-Object InstanceID, RemoteAddress   #
10.0.60.20 only
New-Item -ItemType Directory -Force D:\Inventory | Out-Null
icacls D:\Inventory /inheritance:r /grant:r 'SYSTEM:(OI)(CI)F' 'Administrators:(OI)(CI)F'
'current', 'history', 'sources', 'findings' | ForEach-Object { New-Item -ItemType
Directory -Force "D:\Inventory\$_" | Out-Null }
icacls D:\Inventory
# From SBD-ADMIN-PC01: the same management path must fail
Test-NetConnection SBD-MGMT01.sbd.test -Port 5985   # expected: TcpTestSucceeded False
```

WHAT SUCCESS LOOKS LIKE SBD-MGMT01 sits in the Servers OU with the hypervisor's time synchronization off and time from a domain controller, reports to Ring 3 on SBD-WSUS01, forwards its security events, accepts management only from 10.0.60.20 and refuses SBD-ADMIN-PC01, runs no extra role or agent, and holds an Inventory folder that only SYSTEM and Administrators can open.

Your record

Evidence to keep: Keep the OU placement, the time source, the update policy values and WSUS listing, the firewall scope, the refused connection from SBD-ADMIN-PC01, and the Inventory folder's permissions.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why scope the existing Remote Desktop and remote management rules instead of adding a narrow new rule?

REMEMBER In the Servers OU: hardened, tiered and patched from first boot. Management from SBD-ADMIN-SRV01 only — proven by a refused attempt. A record folder for administrators alone, with history and hashes.

NIST · SP 800-53 Rev. 5 security controls (CM-8 System Component Inventory)

Microsoft · Configure Windows Firewall rules with Group Policy

Microsoft · Icacls

© 2026 Secure By Design Team LLC.

LESSON 03 / ABOUT 30 MIN

Build the living inventory

> **WHY THIS MATTERS** Course 1.13's fleet inventory was a snapshot of fifty PCs; SBD-MGMT01 makes the whole network's inventory a living record. Every component gets its place, ring, update source, baseline and owner, kept current by the procedures that change it.

In this lesson you will

- Load every component into the inventory from the records the path already keeps.
- Define how the inventory stays current and who owns it.
- Publish the first version with its hash.

Follow the build

1. Start from the records the path already keeps rather than from memory: course 1.13's fleet inventory for the 50 PCs (names, image ancestry, certificate thumbprints, sample status), the service-path register and Inside the Network for names, addresses and locations, course 1.14's ring table for the update rings, course 1.9's snapshot catalog for SBD-REPO01, and each course's baseline set for the latest baseline date.
2. Load the inventory with every component the boundary lists — the fleet, the servers (SBD-REPO01 included, with its current snapshot as its update source), the admin plane, the core, the access switch and the hypervisor hosts — each with every column filled: the CM-8 component inventory, in one place. Add the planned systems as planned.
3. Define currency: every build, move and retirement updates the inventory as part of its own procedure (CM-8(1)), so each later course's build lesson adds its rows, and a monthly reconciliation compares the inventory against the directory, the DHCP leases and the WSUS computer list — drift between them is a finding.
4. Name the owner — a role, with a named deputy — who approves each change, and publish the first version: copy it into history with its hash and log the change. Course 2.2 shows how a record like this one joins the Body of Evidence for the configuration-management controls.

Check your result

```
# In a session on SBD-MGMT01 from SBD-ADMIN-SRV01, once the record is filled in
$inv = Import-Csv D:\Inventory\current\inventory.csv
$inv | Group-Object Status | Select-Object Name, Count
$inv | Group-Object Name | Where-Object Count -gt 1 | Select-Object Name   # expected:
nothing counted twice
$inv | Where-Object { $_.Status -eq 'active' -and (-not $_.Owner -or -not $_.UpdateSource
-or -not $_.Location) } | Select-Object Name   # expected: nothing
$stamp = Get-Date -Format yyyy-MM-dd
Copy-Item D:\Inventory\current\inventory.csv "D:\Inventory\history\inventory-$stamp.csv"
Get-FileHash -Algorithm SHA256 "D:\Inventory\history\inventory-$stamp.csv" | Export-Csv
D:\Inventory\history\hashes.csv -Append -NoTypeInformation
```

> **WHAT SUCCESS LOOKS LIKE** The inventory lists every component in the boundary exactly once with every column filled, planned systems are marked planned, the update-on-change rule and the monthly reconciliation are written with an owner and deputy, and the first version is in history with its hash.

Your record

Evidence to keep: Keep the populated inventory export, the status counts and completeness checks, the currency rule with its owner and cadence, and the first version's hash.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

What makes an inventory 'living' rather than a stale spreadsheet?

> **REMEMBER** One record of everything, with place, ring, update source, baseline and owner. Every change updates it; a monthly reconciliation keeps it honest. Owned, versioned, and ready to join the Body of Evidence.

NIST · SP 800-53 Rev. 5 security controls (CM-8 System Component Inventory)

Microsoft · Get-FileHash

LESSON 04 / ABOUT 35 MIN

Feed it from what the network already knows

> **WHY THIS MATTERS** The inventory says what should exist; the network's own records say what does. Export the directory's computer objects, the DHCP leases, the WSUS computer list, SBD-REPO01's records and the baseline sessions through the sessions administrators already use, and file them on SBD-MGMT01 with hashes, in one shape lesson 5 can compare.

In this lesson you will

- Export each discovery source through an existing administration path.
- Know what each source can and can't prove.
- File the exports on SBD-MGMT01 with hashes, in one comparable shape.

Follow the build

1. Read the directory over LDAP from SBD-ADMIN-SRV01: every computer object's name, operating system, distinguished name (which gives its OU) and lastLogonTimestamp. Use an LDAP search rather than the Active Directory module, whose Web Services port the controllers open only to AdminJumpBox01, and read lastLogonTimestamp as a sign of staleness, not of last use: it is updated only when the stored value is 9 to 14 days old.
2. Export the leases on SBD-DHCP01 in a session from SBD-ADMIN-SRV01: every lease in the VLAN 80 scope with its address, client ID (the MAC address), host name, state and expiry. A lease shows what is plugged in now, joined or not; course 1.7's DHCP audit log shows what came and went between exports.
3. Export SBD-WSUS01's computer list the same way: each computer's name, address, last contact, last status report and requested ring. WSUS's cleanup removes computers silent for 30 days (course 1.14), so a machine missing there has been quiet for a month or more.
4. Copy SBD-REPO01's records over SSH as course 1.9 does, and count its web server's access log by client address to see which Linux servers actually take packages. List the baseline sessions in the evidence folders you hold, by host and date; Tier 0 reports its hosts' dates through the change record, because its evidence stays on AdminJumpBox01.
5. File the set: hash every export into a dated list, copy both into D:\Inventory\sources on SBD-MGMT01 through a remoting session, and reduce every name to one key — the short host name, so a lease's SBD-PC07.sbd.test and the directory's SBD-PC07 meet. Take all the sources on the same day; sources taken weeks apart disagree about machines that simply moved.

Check your result

```
# On SBD-ADMIN-SRV01 (Tier 1): one dated folder for this month's sources
$d = "C:\Evidence\management-server\sources\$(Get-Date -Format yyyy-MM-dd)"; New-Item -
ItemType Directory -Force "$d\repo01" | Out-Null
$s = [adsisearcher]'(objectCategory=computer)'; $s.PageSize = 500
$s.FindAll() | ForEach-Object { $p = $_.Properties; [pscustomobject]@{ Name = "$
($p.name)"; OS = "$($p.operatingsystem)"; DN = "$($p.distinguishedname)"; LastLogon = if
("$($p.lastlogontimestamp)") { [datetime]::FromFileTime([long]"$
($p.lastlogontimestamp)") } } } | Export-Csv "$d\directory.csv" -NoTypeInformation
Invoke-Command SBD-DHCP01 { Get-DhcpServerv4Lease -ScopeId 10.0.80.0 -AllLeases | Select-
Object IPAddress, ClientId, HostName, AddressState, LeaseExpiryTime } | Export-Csv "$d\
dhcp-leases.csv" -NoTypeInformation
```

```
Invoke-Command SBD-WSUS01 { Get-WsusComputer -All | Select-Object FullDomainName,
IPAddress, LastSyncTime, LastReportedStatusTime, RequestedTargetGroupName } | Export-Csv
"$d\wsus-computers.csv" -NoTypeInformation
scp -r <your-account>@SBD-REPO01.sbd.test:/srv/repo-records/* "$d\repo01\"
# On SBD-REPO01: who fetched packages
sudo awk '{print $1}' /var/log/httpd/access_log | sort | uniq -c
# Back on SBD-ADMIN-SRV01: hash the set and file it on SBD-MGMT01
Get-ChildItem $d -Recurse -File | Get-FileHash -Algorithm SHA256 | Export-Csv "$d-
hashes.csv" -NoTypeInformation
$m = New-PSSession SBD-MGMT01; Copy-Item $d, "$d-hashes.csv" -Destination D:\Inventory\
sources\ -ToSession $m -Recurse
```

WHAT SUCCESS LOOKS LIKE Each source export exists for the same date with the columns lesson 5 needs, the hash list matches the files copied to SBD-MGMT01, every name reduces to one key, and a note says what each source can and can't prove.

Your record

Evidence to keep: Keep the dated source exports, their hash list, the REPO01 access-log summary, the baseline-session list, and the note on what each source proves.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why read the directory over LDAP instead of with Get-ADComputer from SBD-ADMIN-SRV01?

REMEMBER Five sources, gathered through paths administrators already use. Each proves something narrow; together they test the record. Same day, one name key, hashed and filed on SBD-MGMT01.

NIST · SP 800-53 Rev. 5 security controls (CM-8 System Component Inventory)

Microsoft · DirectorySearcher class (LDAP searches of Active Directory)

Microsoft · Last-Logon-Timestamp attribute

Microsoft · The lastLogonTimestamp attribute may not be accurate

Microsoft · Get-DhcpServerv4Lease

Microsoft · Get-WsusComputer

Microsoft · The WSUS Server Cleanup Wizard

Microsoft · Copy-Item (copying to and from a remote session)

Microsoft · Get-FileHash

LESSON 05 / ABOUT 35 MIN

Reconcile the inventory and prove it

> **WHY THIS MATTERS** Compare the record with what the network reports, classify every difference, and prove the comparison catches drift before trusting a clean result. Then take SBD-MGMT01's first SCAP baseline and hand a reconciled inventory — and the monthly habit — to the courses that follow.

In this lesson you will

- Reconcile recorded and discovered components and classify every difference.
- Prove the reconciliation catches drift with a deliberate test.
- Baseline SBD-MGMT01 and hand off the reconciled record.

Follow the build

1. On SBD-MGMT01, compare the inventory's active rows with each source by the name key: the domain-joined rows against the directory, the VLAN 80 rows against the leases, and the Windows rows against WSUS's computer list and its rings. In a source but not the record is an unrecorded component; recorded but absent where it should appear is a gap; present in both with a different value — a ring, an OU, an address — is drift.
2. Prove the comparison works before trusting a clean result: run it against a test copy of the inventory with one PC's row removed — the PC must appear as found in the directory but not in the record — and against another copy with one ring changed, which the ring comparison must report. Test copies are never published.
3. Classify every real difference and close it: correct the record through the change log, bring the component under management or remove it, or record the explanation — a PC switched off for a week has no lease and isn't missing. An unrecorded device is reported to the system owner the day it is found (CM-8(3)), and drift that keeps coming from one procedure is fixed in that procedure.
4. Take SBD-MGMT01's first SCAP baseline with course 1.4's SCC and benchmark versions, run locally in a Tier 1 session from SBD-ADMIN-SRV01, classify its open items as course 1.4 taught, and enter the date in its own inventory row.
5. Hand off: publish the reconciled version with its hash, set the monthly reconciliation in the week after course 1.14's cycle closes so the WSUS list is fresh, and note what later courses add — course 1.18 turns the VLAN 40 servers' planned rows active as it builds them, and course 1.19 carries D:\Inventory into the recovery set.

Check your result

```
# On SBD-MGMT01 (session from SBD-ADMIN-SRV01): the newest sources against the active
record
$src = (Get-ChildItem D:\Inventory\sources -Directory | Sort-Object Name | Select-Object -
Last 1).FullName
$short = { $args[0] -replace '\..*$' }
$active = Import-Csv D:\Inventory\current\inventory.csv | Where-Object Status -eq 'active'
$directory = (Import-Csv "$src\directory.csv").Name
$leases = Import-Csv "$src\dhcp-leases.csv" | Where-Object HostName | ForEach-Object { &
$short $_.HostName }
$rings = Import-Csv "$src\wsus-computers.csv" | ForEach-Object { '{0}|{1}' -f (& $short
$_.FullDomainName), $_.RequestedTargetGroupName }
```

```
Compare-Object ($active | Where-Object Domain -eq 'sbd.test').Name $directory   # <=
recorded only, => directory only
Compare-Object ($active | Where-Object VLAN -eq '80').Name $leases
Compare-Object ($active | Where-Object UpdateSource -like 'Ring*' | ForEach-Object { '{0}|
{1}' -f $_.Name, $_.UpdateSource }) $rings
# Prove it: with SBD-PC07's row left out of a test copy, SBD-PC07 must show as directory
only
Compare-Object ($active | Where-Object { $_.Domain -eq 'sbd.test' -and $_.Name -ne 'SBD-
PC07' }).Name $directory
```

> **WHAT SUCCESS LOOKS LIKE** Every difference between the record and the sources is listed and classified, the deliberate tests were caught, unrecorded components were reported to the system owner, SBD-MGMT01 has its first baseline, and the reconciled version is published with its hash and the next reconciliation date.

Your record

Evidence to keep: Keep the comparison output, the classified findings with their resolutions, the tests that proved detection, the SCC results with classified findings, the reconciled version's hash, and the handoff note.

RECORD	YOUR OBSERVATION
Host(s) and time collected	
Observed result	
Differences from the plan	
Evidence file names	
Open items or findings	

Think it through

Why run the comparison against a deliberately broken copy first?

> **REMEMBER** Recorded against discovered: unrecorded, gap or drift — each classified. A planted difference, caught, makes a clean result mean something. Reconciled, baselined and handed on — every month from now on.

NIST · SP 800-53 Rev. 5 security controls (CM-8 System Component Inventory)

Microsoft · Get-FileHash

DISA · SCAP content and the SCAP Compliance Checker (SCC)

DISA · SRG/STIG tools (SCC and STIG Viewer)

Course review and sign-off

Before moving on, confirm each lesson left evidence behind — and that what remains open is written down, not remembered.

Lesson completion

LESSON	EVIDENCE FILED	DATE
01 Design the inventory and its keeper	☐ yes	
02 Build and harden SBD-MGMT01	☐ yes	
03 Build the living inventory	☐ yes	
04 Feed it from what the network already knows	☐ yes	
05 Reconcile the inventory and prove it	☐ yes	

Open items carried forward

Next in the learning path

That completes course 1.15. Courses 1.16 and 1.17, endpoint protection and credentialed scanning, are coming soon; course 1.18 stands up the business services and turns their planned rows in this inventory active.

> **KEEP IT HONEST** Lesson completion records learning activity; it is not a system authorization decision. Your observations, not the sample values, establish what happened in your environment.

© 2026 Secure By Design Team LLC.